

Optimal simulation of self-verifying automata by deterministic automata

1. Introduzione

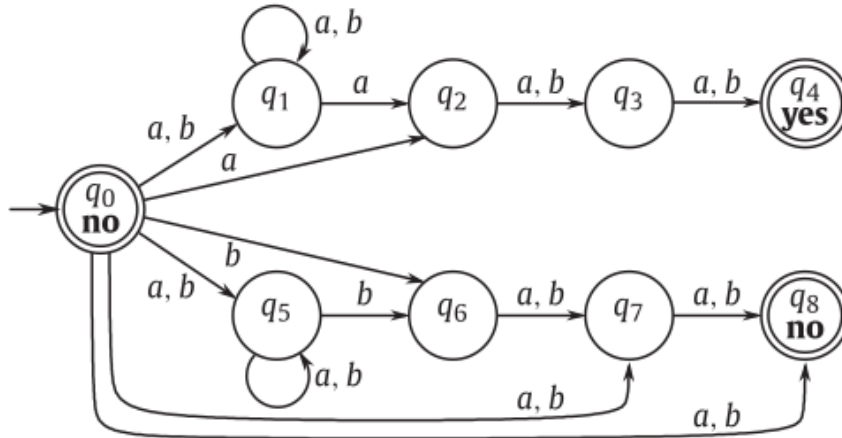
Gerarchia di Chomsky e classe dei **linguaggi regolari**:

- come sono riconosciuti (brevissimo);
- equivalenza tra DFA e NFA (brevissimo).

Introduzione ai **SVFA**:

- NFA con nodi speciali (**non determinismo simmetrico**);
- perché il nome self-verifying.

Automa per il riconoscimento del linguaggio L_3 (ricorda che puoi togliere uno stato).



2. Definizione

Definizione formale di un SVFA:

- tupla $A = (Q, \Sigma, \delta, q_0, F^a, F^r)$;
- proprietà:
 - almeno una computazione che accetta/rifiuta;
 - non ci sono risposte discordanti;
 - riprendi il discorso sul nome self-verifying;
- linguaggio riconosciuto, rifiutato e relazione $L^a(A) = (L^r(A))^C$.

Numero di stati:

- se mi vengono dati due NFA;
- se mi viene dato un SVFA;
- relazione $\max(\text{nsc}(L), \text{nsc}(L^C)) \leq \text{svsc}(L) \leq 1 + \text{nsc}(L) + \text{nsc}(L^C)$.

3. Conversione

Uso della **subset construction** per l'equivalenza.

Togliamo tutti gli stati non raggiungibili dell'automa A dato.

Definizione di L_q^a e L_q^r di uno stato q come l'insieme delle stringhe accettate/rifiutate da A partendo dallo stato q . Proprietà di:

- **completezza** $L_{q_0}^a \cup L_{q_0}^r = \Sigma^*$;
- **consistenza** $L_{q_0}^a \cap L_{q_0}^r = \emptyset$.

Estendi queste definizioni usando un sottoinsieme α come stato.

Migliorare il **numero di stati** della subset construction: vogliamo trovare il numero di sottoinsiemi raggiungibili usando una **relazione di compatibilità**.

Due stati sono **compatibili** se due computazioni che partono da questi due stati non danno risposte discordanti, ovvero se

$$(L_p^a \cup L_q^a) \cap (L_p^r \cup L_q^r) = \emptyset.$$

Grafo di compatibilità e cricche del grafo come sottoinsiemi. Visto che gli stati presenti in un sottoinsieme α devono essere compatibili allora per il DFA risultante possiamo vedere il numero di queste cricche.

Numero di stati pari a

$$1 + f(n - 1)$$

dove $f(n)$ è il massimo numero di cricche in un grafo di n nodi. La funzione f è tale che

$$f(n) = \begin{cases} 3^{\lfloor n/3 \rfloor} & \text{se } n \equiv 0 \pmod{3} \\ 4 \cdot 3^{\lfloor n/3 \rfloor - 1} & \text{se } n \equiv 1 \pmod{3} \\ 2 \cdot 3^{\lfloor n/3 \rfloor} & \text{se } n \equiv 2 \pmod{3} \end{cases} \quad \forall n \geq 2.$$

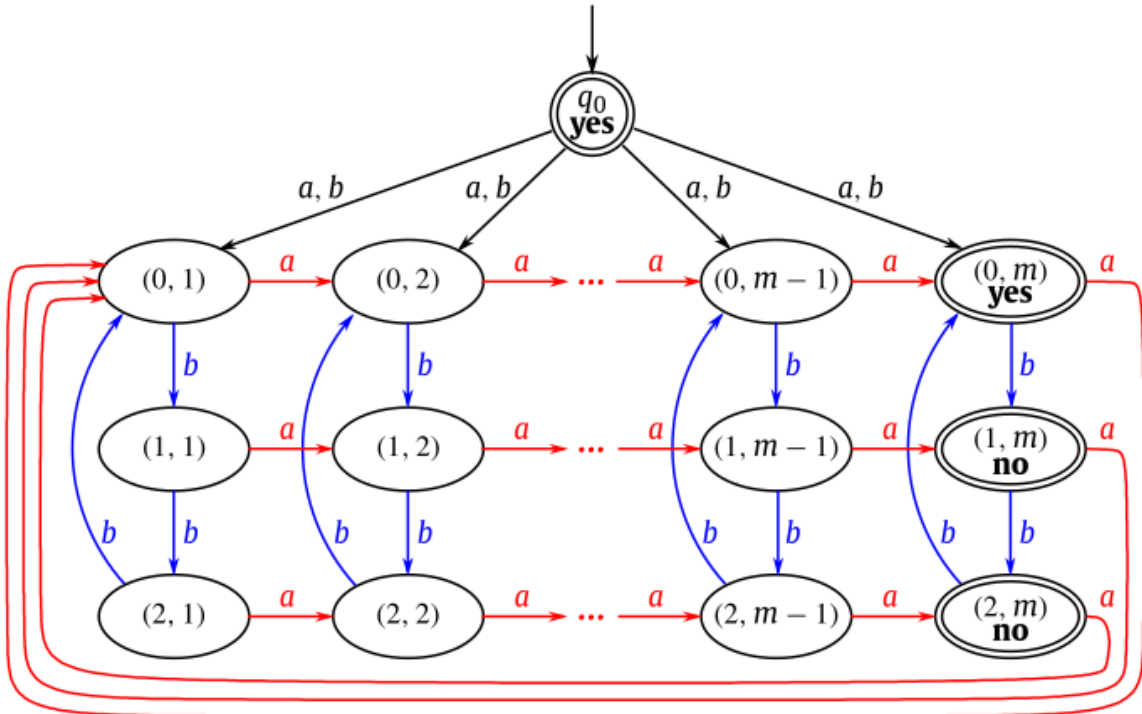
4. Ottimalità

Dimostriamo che per ogni n allora la nostra costruzione è ottima, ovvero che per ogni n esiste un automa di n stati che genera un DFA minimo con esattamente quel numero di stati.

Visto che la funzione $f(n)$ è «periodica», possiamo considerare solo tre «classi» di automi.

Partiamo con automi che hanno un numero di stati pari a

$$n = 1 + 3m \mid m \geq 2.$$



La funzione di transizione di questo automa è tale che

$$\begin{aligned}\delta(q_0, a) &= \delta(q_0, b) = \{(0, 1), \dots, (0, m)\} \\ \delta((i, j), a) &= \begin{cases} \{(i, j+1)\} & \text{se } j < m \\ \{(0, 1)\} & \text{altrimenti} \end{cases} \\ \delta((i, j), b) &= \{(i+1) \bmod 3, j\}.\end{aligned}$$

Cosa accetta questo automa: stringa vuota sicuramente.

Ora prendiamo una stringa $w = \sigma w'$ e chiamiamo m_0 il numero di occorrenze di a in w' . Con il carattere σ in maniera non deterministica ci spostiamo negli stati della prima riga. Con questo stiamo guessando il valore m_0 . Il guess giusto dipende anche dal numero di b :

- se $\#_b \equiv 0 \bmod 3$ accetto;
- altrimenti rifiuto.

Notiamo che il counter però viene azzerato quando passiamo dalla colonna di destra a quella di sinistra.

Quindi accettiamo in due casi:

- se abbiamo meno di m caratteri pari ad a e 0 caratteri b modulo 3;
- se la stringa w' ha un suffisso di m caratteri a con 0 caratteri b modulo 3.

Dimostrazione che è self-verifying.

Prendiamo l'insieme

$$Q' = \{(x_1, 1), \dots, (x_m, m)\} \mid x_1, \dots, x_m \in \{0, 1, 2\}.$$

Stiamo scegliendo una cella per ogni colonna della griglia. Se applichiamo una a o una b ad un insieme di Q' otteniamo ancora un elemento di Q' . Quando noi da q_0 ci spostiamo in nella prima riga della griglia noi siamo in uno stato di Q' . Visto che stiamo selezionando solo un elemento di ogni colonna, non possiamo selezionare due stati dell'ultima, quindi siamo self-verifying.

Dimostrazione che è minimo.

Trasformiamo gli stati di Q' nei vettori $(x_1, \dots, x_m) \in \{0, 1, 2\}^m$. Notiamo che

$$\begin{aligned}\delta((x_1, \dots, x_m), a) &= (0, x_1, \dots, x_{m-1}) \\ \delta((x_1, \dots, x_m), b) &= ((x_1 + 1) \bmod 3, \dots, (x_m + 1) \bmod 3).\end{aligned}$$

Tutti questi stati sono raggiungibili:

- raggiungiamo lo stato $(0, \dots, 0)$ da qualsiasi altro stato se applichiamo la stringa a^m allo stato corrente, ovvero

$$\delta((x_1, \dots, x_m), a^m) = (0, \dots, 0);$$

- raggiungiamo qualsiasi stato da $(0, \dots, 0)$ usando la relazione

$$\delta((0, x_2 - x_1, \dots, x_m - x_1), b^{x_1}) = (x_1, \dots, x_m)$$

e usando una dimostrazione per induzione.

Questi stati sono $1 + 3^m$. Inoltre, sono tutti distinguibili tra loro.

Infatti, se prendiamo due stati che hanno $x_i \neq y_i$. Se partiamo dagli stati $(0, i)$ e $(1, i)$ allora questi due non sono compatibili con a^{m-i} . Idem se prendiamo la coppia $(0, 2)$ e anche $(1, 2)$ se

aggiungiamo una b alla stringa. Si può fare lo stesso discorso con lo stato q_0 e gli altri accettanti/rifiutanti.

Se invece $n = 3m + 2 \mid m \geq 2$ si aggiunge lo stato $(3, 1)$, mentre con $n = 3m \mid n \geq 2$ si toglie lo stato $(2, 1)$. In ogni caso, si ottengono i valori di $f(n - 1)$.

Con $n = 1$ e $n = 2$ si hanno SVFA equivalenti a DFA, quindi il numero di stati è definito alla funzione

$$g(n) = \begin{cases} 1 + 3^{(n-1)/3} & \text{se } n \equiv 1 \pmod{3} \wedge n \geq 4 \\ 1 + 4 \cdot 3^{(n-2)/3-1} & \text{se } n \equiv 2 \pmod{3} \wedge n \geq 5 \\ 1 + 2 \cdot 3^{n/3-1} & \text{se } n \equiv 0 \pmod{3} \wedge n \geq 3 \\ n & \text{se } n \leq 2 \end{cases}$$